CSED233 · 데이터구조 · 포스텍

Ch3. Algorithm Analysis — Big-O & 점근 분석

1. Algorithm(알고리즘) 비교: 왜 필요한가

이론 & 배경

- **Algorithm**: 유한 시간 내 문제를 해결하는 step-by-step 절차.
- 같은 문제에 여러 알고리즘 → 어느 것이 더 나은가?
- **비교 척도 = 효율성(Efficiency)**
  - **Time complexity(시간복잡도)**: 실행 시간
  - **Space complexity(공간복잡도)**: 메모리 사용량

두 가지 비교 방법

방법	장점	한계
Empirical(경험적 분석) (실제 실행)	구체적 성능 수치	구현 비용, 입력 의존, 동일 HW/SW 필요
Theoretical(이론적 분석) (이론 분석)	HW/SW 독립, 모든 입력 고려	상수항 무시로 실제와 괴리 가능

- 이론 분석: 입력 크기 n에 대한 running time/space를 **함수**로 표현.

2. 점근 분석 (Asymptotic Analysis)

원리

"극한 행동(limiting behavior)만 본다" — $n \to \infty$ 관점.

1. **Lower-order term 무시**

- 예: $f(n) = n^2 + 3n \to n$ 커지면 $3n$은 n^2에 비해 미미

- → $f(n) \sim n^2$ (asymptotically equivalent)

2. **상수 무시**

- 예: $6N^2$ vs $20N^2$ → $N \gg 20$ 이면 N^2 자체가 증가 주체
- 상수 6, 20은 중요치 않음

Best(최선) / Worst(최악) / Average(평균)

Case	의미	분석 용이성
Best(최선) (Ω)	"적어도 이만큼"	참고용
Average(평균) (Θ)	"보통 이만큼"	결정 어려움
Worst(최악) (O)	"최악 이만큼"	**쉬움, 실시간 앱 critical**

- 이론 분석은 **대부분 worst case** 집중.
- O/Ω/Θ는 상한·하한·타이트 경계 **표기**, Best/Avg/Worst는 입력 사례 구분 — 각 case의 함수에 표기를 적용 (위 괄호는 자주 쓰는 대응)

3. Big-O(빅오) 정의 & 성질

정의

> $T(n) = O(f(n)) \iff \exists\ c > 0, \exists\ n_0 \geq 1$ s.t. $T(n) \leq c \cdot f(n)$ for all $n \geq n_0$.

- **Upper bound**(증가율 상한)
- Big-Ω: $T(n) \geq c \cdot f(n)$ (∃ c>0, ∃ $n_0 \geq 1$, 모든 $n \geq n_0$) → **lower bound**
- Big-Θ: O와 Ω 동시 만족 → **asymptotically equal**

대표 예시 — 유도 과정 포함

예 1: $T(n) = (n+1)^2$ 이 $O(n^2)$ 인가?

$(n+1)^2 = n^2 + 2n + 1$
$\leq n^2 + 2n^2 + n^2$ ($\because$ $n \geq 1$ 이면 $1 \leq n^2$, $2n \leq 2n^2$)

$= 4n^2$

→ c = 4, $n_0 = 1$ 로 성립 ⟹ $O(n^2)$ ✓

기타:

- $3n^3 \in O(n^3)$ (tight) ↔ $3n^3 \in O(n^4)$ (loose)
- $3n^3 + 2n^2 + 8 \in O(n^3)$
- $2^{100} \in O(1)$ (상수)
- $3\log n + 5 \in O(\log n)$

4. 성장률 순위 & Properties

Typical Growth Rate (작은 순)

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

Big-O(빅오) 연산 규칙

- **Addition**: $T_1 \in O(f), T_2 \in O(g) \Rightarrow T_1 + T_2 \in O(\max f, g)$
- **Product**: $T_1 \in O(f), T_2 \in O(g) \Rightarrow T_1 \cdot T_2 \in O(f \cdot g)$
- 상수배: $a \cdot T(n) \in O(f(n))$
- 추이성: $T \in O(f), f \in O(g) \Rightarrow T \in O(g)$
- 다항식 차수 $d \rightarrow T(n) \in O(n^d)$

5. 실제 분석 방법

기본 가정

Basic operations = constant time:

- 산술 연산 (fixed-width)
- Assignment
- Array(배열) index 접근
- 단순 값 비교

조합 규칙:

- 연속문 → **합**
- 루프 → (body cost) × (iteration count)

예시: 3중 행렬 곱셈

```
for i := 1 to n
 for j := 1 to n begin
  C[i,j] := 0
  for k := 1 to n
    C[i,j] := C[i,j] + A[i,k]*B[k,j]
 end
```

유도:

$$T(n) = \sum_{i=1}^{n}\sum_{j=1}^{n}(c_1 + \sum_{k=1}^{n} c_2) = c_1 n^2 + c_2 n^3$$

→ $T(n) \in O(n^3)$

---

6. 점근 분석의 한계

주의점

- **상수 factor 무시** → 실제로는 상수가 지배할 수 있음
  - 예: $1000n$ vs n^2 → $n < 1000$에선 n^2이 더 빠름
- **메모리 계층(cache/MM/HDD) 무시**
- 연산 많음 ≠ 느림: memory fetch 비용이 실제로는 CPU 연산보다 훨씬 큼

Q1. 루프 안의 if가 일부 iteration에서만 실행될 때 점근 분석은 어떻게 하나?

> 힌트

점근 분석의 기본 기준은 worst case다.

정답

개념은 힌트 참고.

if가 일부 반복에서만 실행돼도, **worst case 기준**으로는 매 반복 실행되는 입력을 기준으로 분석한다 (실현 가능한 최악의 실행 경로의 비용).

정답: worst case 기준으로 if가 항상 실행된다고 가정한다.

7. Amortized(분할 상환) Analysis (분할 상환)

동기

- `put(x)` 연산이 가끔 배열 확장 필요 → worst case $O(n)$이지만 매번 그런 건 아님
- **평균 비용**을 n번 call에 대해 계산:

$$\text{amortized cost} = \frac{\sum_{i=1}^{n}(\text{cost of i-th call})}{n}$$

두 버전 비교

Version 1: 크기 +5씩 증가 (상수 증가)

확장은 5번마다 → 5, 10, 15, ... 크기 복사. **총 비용 유도** ($n = 5m$ 번 put):

```
Total = n + (0 + 5 + 10 + ... + 5(m-1))
      = n + 5 · m(m-1)/2
      ≈ n + n²/10                ← ∵ m = n/5

amortized = (n + n²/10)/n = 1 + n/10 = O(n)
```

→ **call당 O(n)**. 나쁨.

Version 2: 크기 × 2 (더블링)

확장은 n이 2의 거듭제곱일 때 → 1, 2, 4, 8, ... 복사. **총 비용** ($n = 2^k$):

```
Total ≈ n + (1 + 2 + 4 + ... + 2^(k-1))
      ≈ n + (2^k - 1)    ← 기하급수 합
      ≈ 2n - 1
```

```
amortized = (2n-1)/n = 2 - 1/n = O(1)
```

→ **call당 O(1)**. 좋음.

교훈

> **상수 증가 → $O(n)$ per call, 더블링 → $O(1)$ per call**. 같은 worst case $O(n)$이라도 amortized 관점에서 전혀 다른 성능.

Summary

개념	핵심
Algorithm(알고리즘)	finite-time step 절차
Empirical(경험적 분석) vs Theoretical(이론적 분석)	실측 vs 수학 분석
Best(최선)/Avg/Worst(최악)	Ω / Θ / O
Big-O(빅오)	상한 (upper bound)
Amortized(분할 상환)	여러 call의 평균 (doubling 효과)